

Decision and Risk Analysis

Lecture 7: **Approximating the Posterior**

Grid Approximation, MCMC, and RStan

DAEN 427 / ISEN 427 / ISEN 627 - Fall 2026

Alexander Abuabara



Learning goals

1. Explain why posterior approximation is needed.
2. Implement grid approximation in R.
3. Explain the basic idea of MCMC and RStan.
4. Read basic MCMC diagnostics: trace plots, autocorrelation, effective sample size, and R-hat.
5. Use posterior draws for practical risk decisions.

simple exact model → grid approximation →
→ MCMC → RStan → diagnostics → decisions

Example

A supplier says that most parts pass inspection.

We test 10 parts and 9 pass.

Unknown

$$\pi = P(\text{part passes})$$

Prior

$$\pi \sim \text{Beta}(2, 2)$$

This prior is centered near 0.5,
but still uncertain.

Data model

$$Y \mid \pi \sim \text{Bin}(10, \pi), \quad Y = 9$$

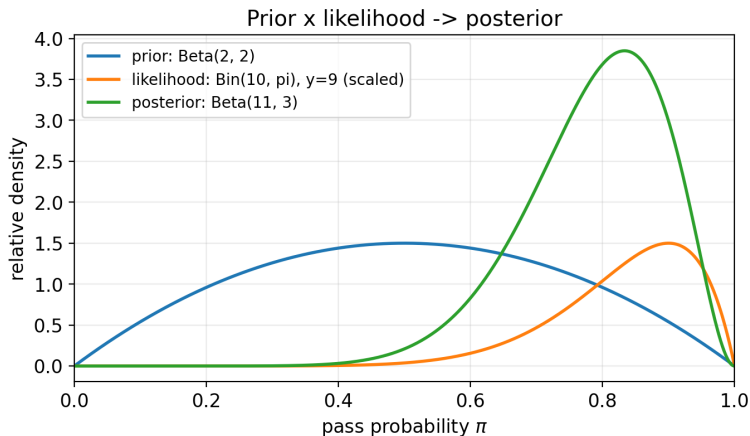
Risk question

$$P(\pi < 0.70 \mid Y = 9)?$$

If this is high,
inspect more or reject the supplier.

Bayesian update

$$\pi \mid Y = 9 \sim \text{Beta}(2 + 9, 2 + 1) = \text{Beta}(11, 3).$$



Let us use a simple model because the exact posterior is known, and the approximation methods can be checked.

Why approximate at all?

For simple models, we can sometimes write the posterior exactly:

$$f(\theta | y) = \frac{f(\theta)L(\theta | y)}{f(y)}.$$

For realistic models, θ may contain many parameters:

$$\theta = (\theta_1, \theta_2, \dots, \theta_k).$$

Then the normalizing constant may require a hard integral:

$$f(y) = \int \cdots \int f(\theta)L(\theta | y)d\theta.$$

Produce draws $\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(N)}$ that behave like posterior draws.

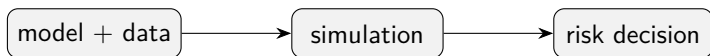
Two approximation ideas

1. Grid approximation

- Try many possible parameter values.
- Score each value by $\text{prior} \times \text{likelihood}$.
- Normalize scores into probabilities.
- Sample from the discrete grid.

2. MCMC

- Build a dependent sequence of values.
- Each next value depends on the current value.
- A good chain mimics the posterior.
- RStan handles the sampling algorithm.



Grid approximation algorithm

Goal: approximate $f(\theta \mid y)$ using a finite set of possible values.

1. Define a grid of possible θ values.
2. Evaluate the prior and likelihood at each grid value.
3. Multiply and normalize:

$$\text{posterior probability} = \frac{\text{prior} \times \text{likelihood}}{\sum \text{prior} \times \text{likelihood}}.$$

4. Sample grid values using those posterior probabilities.

Intuition

A grid is like viewing a curve through many vertical slices.
More slices give a clearer picture.

Grid approximation in R

```
# Y | pi ~ Bin(10, pi), pi ~ Beta(2, 2), observed Y = 9
library(tidyverse)

# Step 1: choose grid points
grid_data <- data.frame(pi_grid = seq(0, 1, length = 101))

# Step 2: evaluate prior and likelihood on the grid
grid_data <- grid_data %>%
  mutate(prior = dbeta(pi_grid, 2, 2),
         likelihood = dbinom(9, size = 10, prob = pi_grid))

# Step 3: prior x likelihood, then normalize
grid_data <- grid_data %>%
  mutate(unnormalized = prior * likelihood,
         posterior = unnormalized / sum(unnormalized))
```

Key coding idea: each row is one possible value of π .
The posterior column is a probability distribution across rows.

Grid approximation in R

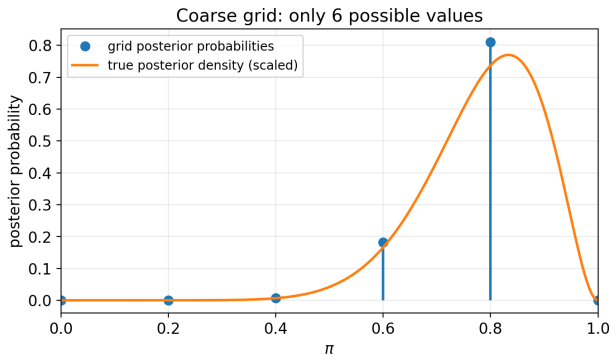
```
set.seed(84735)

# Step 4: sample grid values using posterior probabilities
post_sample <- sample_n(grid_data, size = 10000,
                        weight = posterior, replace = TRUE)

# Use the sample like posterior draws
post_sample %>%
  summarize(mean_pi = mean(pi_grid),
            q05 = quantile(pi_grid, 0.05),
            q95 = quantile(pi_grid, 0.95),
            risk_low_quality = mean(pi_grid < 0.70))
```

After step 4, the posterior sample is the main object. Summaries, intervals, and risk probabilities all come from it.

Coarse grid (few values)

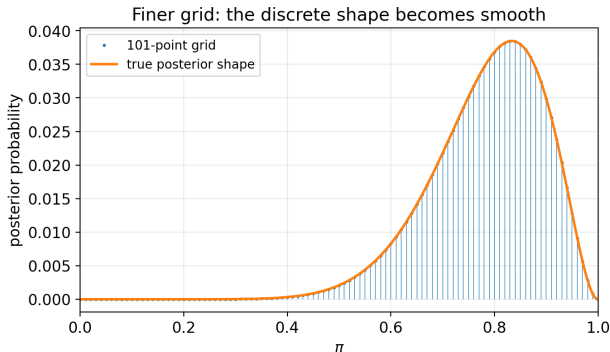


With only 6 grid points, the posterior can only place mass at

0, 0.2, 0.4, 0.6, 0.8, 1.0.

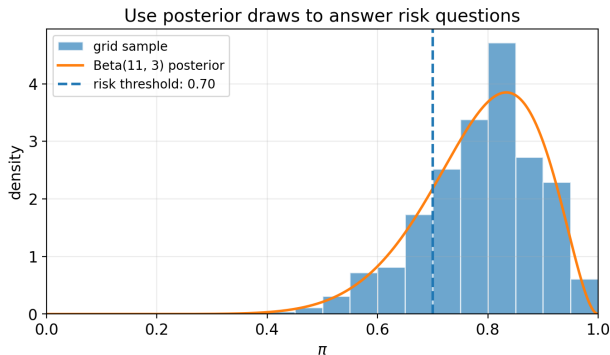
This is easy to understand, but too rigid for careful risk analysis.

Finer grid (approximation improves)



- Using 101 grid values makes the discrete approximation look much closer to the smooth posterior. (usually improves accuracy)
- But also increase computation.

Using posterior draws for a decision



For the supplier example:

$$E(\pi \mid y) \approx 0.786, \quad P(\pi < 0.70 \mid y) \approx 0.20.$$

Decision rule: if $P(\pi < 0.70 \mid y) > 0.10$,
collect more evidence or reject the supplier.

Second example: events per hour

Suppose an industrial engineer monitors events per hour:

- machine stoppages,
- helpdesk tickets,
- safety incidents,
- truck arrivals at a dock.

Let λ be the event rate per hour:

$$Y_i \mid \lambda \sim \text{Pois}(\lambda), \quad \lambda \sim \text{Gamma}(3, 1).$$

We observe two one-hour counts:

$$y = (2, 8), \quad \sum y_i = 10.$$

Exact conjugate result:

$$\lambda \mid y \sim \text{Gamma}(3 + 10, 1 + 2) = \text{Gamma}(13, 3).$$

Gamma-Poisson grid approximation in R

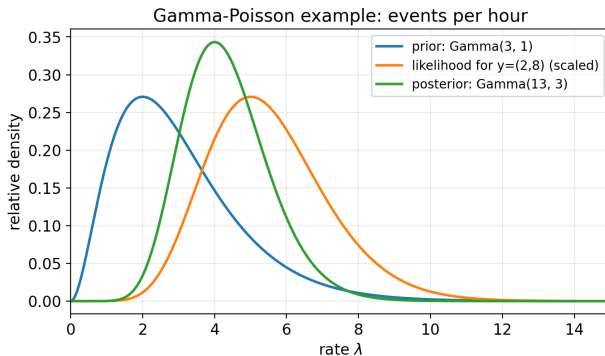
```
# Event count example: Y = c(2, 8)
grid_data <- data.frame(lambda_grid = seq(0, 15, length = 501))

grid_data <- grid_data %>%
  mutate(prior = dgamma(lambda_grid, shape = 3, rate = 1),
         likelihood = dpois(2, lambda_grid) *
                     dpois(8, lambda_grid),
         unnormalized = prior * likelihood,
         posterior = unnormalized / sum(unnormalized))

set.seed(84735)
post_sample <- sample_n(grid_data, size = 10000,
                        weight = posterior, replace = TRUE)

post_sample %>%
  summarize(mean_lambda = mean(lambda_grid),
            risk_high_rate = mean(lambda_grid > 6))
```

Gamma-Poisson

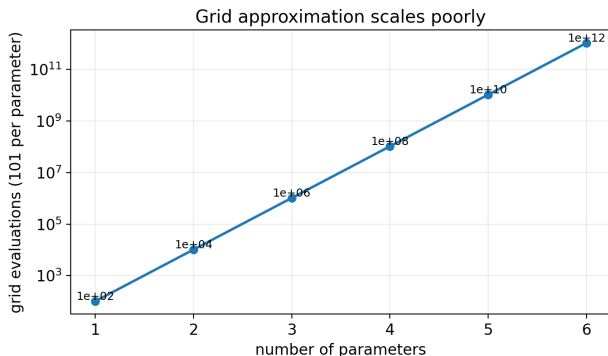


For $\lambda \mid y \sim \text{Gamma}(13, 3)$:

$$E(\lambda \mid y) \approx 4.33, \quad P(\lambda > 6 \mid y) \approx 0.09.$$

Decision use: decide whether capacity for more than 6 events/hour is necessary.

Grid limitation (curse of dimensionality)



If one parameter uses 101 grid values, then k parameters require 101^k grid evaluations. For 5 parameters, that is more than 10^{10} .

Main limitation: Grid approximation is intuitive, but it does not scale well to many parameters.

MCMC

Grid approximation tries to evaluate many locations in advance.
MCMC instead makes a chain:

$$\theta^{(1)} \rightarrow \theta^{(2)} \rightarrow \theta^{(3)} \rightarrow \dots \rightarrow \theta^{(N)}.$$

Markov property

The next draw depends on the current draw, not the full past:

$$f(\theta^{(i+1)} \mid \theta^{(1)}, \dots, \theta^{(i)}, y) = f(\theta^{(i+1)} \mid \theta^{(i)}, y).$$

A good chain spends more time in plausible posterior regions and less time elsewhere.

MCMC versus grid approximation

	Grid approximation	MCMC
Sample type	Independent draws from a discrete approximation	Dependent draws in a chain
Main strength	Very intuitive	Scales better to complex models
Main weakness	Curse of dimensionality	Requires diagnostics
Student image	Try every grid point	Walk around plausible regions

MCMC draws are not independent. We must check whether the chain mixed well enough to trust the approximation.

Stan declarations basic pattern

A Stan declaration tells us three things:

`type<constraints> name;`

Example:

```
real<lower=0, upper=1> theta;
```

Part	Meaning	Example
real	decimal-valued number	0.35, 2.1, -4.7
<lower=0, upper=1>	allowed range	between 0 and 1
theta	variable name	unknown probability

Mathematically: $0 \leq \theta \leq 1$.

Why use constraints?

Constraints keep the model consistent with the meaning of the variable.

```
real<lower=0, upper=1> theta;
```

This is appropriate when θ is a probability:

$$\theta = P(\text{component is defective}).$$

A probability cannot be negative and cannot exceed 1.

Without the constraint:

```
real theta;
```

Stan would allow impossible values such as -0.20 or 1.40 .

Common Stan declarations

Stan declaration	Plain English	Typical use
<code>real mu;</code>	any decimal number	mean or intercept
<code>real<lower=0> sigma;</code>	nonnegative decimal	standard deviation
<code>real<lower=0,upper=1> p;</code>	probability	defect or success rate
<code>int<lower=1> N;</code>	integer at least 1	sample size
<code>array[N] int<lower=0,upper=1> y;</code>	N zeros/ones	binary outcomes
<code>vector[N] x;</code>	N decimal values	predictor values

Important: angle brackets `< >` do **not** mean “less than” here. They declare restrictions.

`real<lower=0> sigma;` means $\sigma \geq 0$.

RStan model structure: Beta-Binomial

The Stan code is a direct translation of the model:

$$Y \mid \pi \sim \text{Bin}(10, \pi), \quad \pi \sim \text{Beta}(2, 2).$$

```
library(rstan)
library(bayesplot)

bb_model <- "
data {
  int<lower = 0, upper = 10> Y;
}
parameters {
  real<lower = 0, upper = 1> pi;
}
model {
  pi ~ beta(2, 2);
  Y ~ binomial(10, pi);
}
"
```

Data are supplied from R. Parameters are unknowns.
The model block contains prior and likelihood.

RStan simulation command

```
bb_sim <- stan(model_code = bb_model,  
              data = list(Y = 9),  
              chains = 4,  
              iter = 5000 * 2,  
              seed = 84735)
```

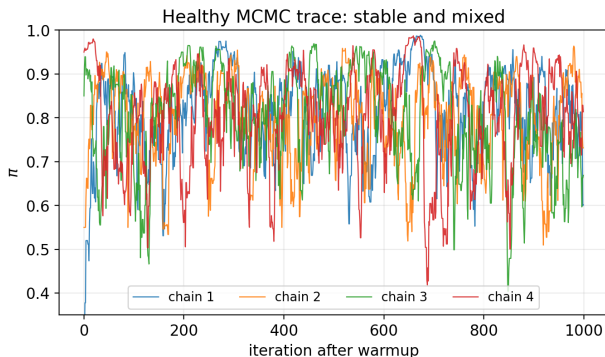
Arguments

- `model_code`: Stan model string.
- `data`: observed data list.
- `chains`: parallel chains.
- `iter`: iterations per chain.

Warmup / burn-in

- The first half is usually warmup.
- Warmup lets the algorithm adapt.
- The retained draws approximate the posterior.

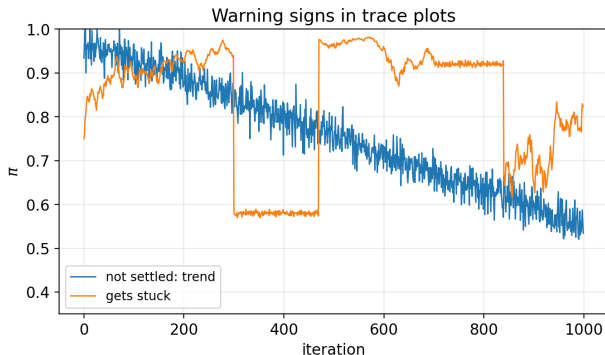
Trace plots: first diagnostic



A healthy trace plot looks stable and noisy, with chains exploring the same range.

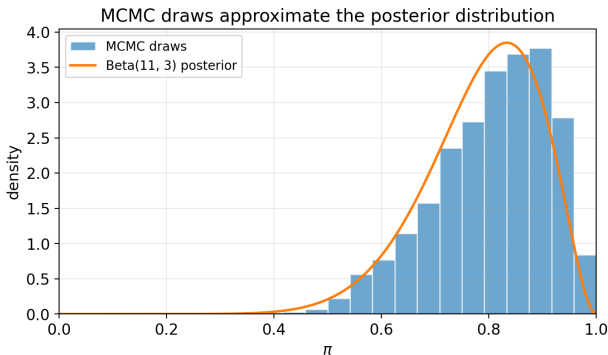
Plain-language test: does each chain wander around the same posterior region without a trend or long pause?

Trace plots: warning signs



- A trend suggests the chain has not settled.
- A flat section suggests the chain got stuck.
- Either problem can give misleading posterior summaries.

Posterior distribution from MCMC draws



After diagnostics look acceptable, summarize retained draws just like a grid sample:

mean, credible interval, $P(\text{bad outcome} \mid y)$.

Diagnostic code in R

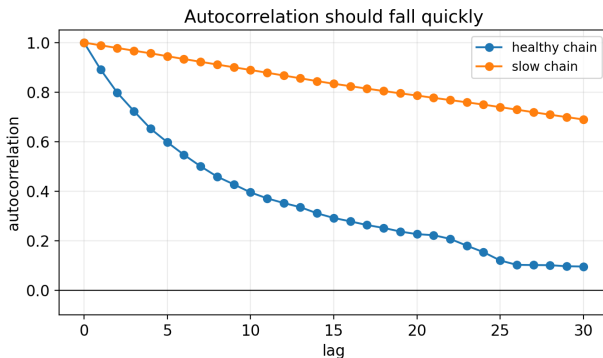
```
# Visual diagnostics
mcmc_trace(bb_sim, pars = "pi")
mcmc_dens_overlay(bb_sim, pars = "pi")
mcmc_acf(bb_sim, pars = "pi")

# Numerical diagnostics
neff_ratio(bb_sim, pars = "pi")
rhat(bb_sim, pars = "pi")
```

How to read the diagnostics

- Trace plot: stable, noisy, no trend, no long sticking.
- Density overlay: parallel chains should look similar.
- Autocorrelation: should drop quickly.
- Effective sample size ratio: larger is better.
- R-hat: should be close to 1; values above about 1.05 raise concern.

Autocorrelation: fast versus slow mixing



Strong autocorrelation means nearby draws are very similar.

Risk-analysis implication

A chain with 20,000 highly correlated draws may contain much less information than 20,000 independent draws.

RStan model structure: Gamma-Poisson

```
gp_model <- "  
data {  
  array[2] int<lower = 0> Y;  
}  
parameters {  
  real<lower = 0> lambda;  
}  
model {  
  lambda ~ gamma(3, 1);  
  Y ~ poisson(lambda);  
}  
"  
  
gp_sim <- stan(model_code = gp_model,  
              data = list(Y = c(2, 8)),  
              chains = 4,  
              iter = 5000 * 2,  
              seed = 84735)
```

Only the likelihood and prior change. The workflow stays the same.

Simple workflow

1. Write the data model and prior.
2. Use grid approximation if there are only one or two parameters.
3. Use MCMC/RStan for more realistic models.
4. Always inspect diagnostics before making decisions.
5. Turn posterior draws into risk quantities:

$$P(\text{failure rate too high} \mid y), \quad E(\text{loss} \mid y).$$

The output is not just an estimate.

It is a distribution of plausible values for risk calculations.

Mini-exercise: change the supplier example

Try a more skeptical prior and fewer successes:

$$Y \mid \pi \sim \text{Bin}(10, \pi), \quad \pi \sim \text{Beta}(3, 8), \quad Y = 2.$$

```
grid_data <- data.frame(pi_grid = seq(0, 1, length = 201)) %>%  
  mutate(prior = dbeta(pi_grid, 3, 8),  
         likelihood = dbinom(2, 10, pi_grid),  
         unnormalized = prior * likelihood,  
         posterior = unnormalized / sum(unnormalized))  
  
post_sample <- sample_n(grid_data, 10000,  
                        weight = posterior, replace = TRUE)
```

Questions. What is $P(\pi < 0.30 \mid y)$? Would you approve the supplier?

Bayesian posterior analysis does not require a closed-form posterior. It requires a trustworthy approximation.

- Grid approximation is the most transparent first approach.
- MCMC is the scalable approach for more complex models.
- Diagnostics are part of the analysis, not an optional extra.
- Posterior draws become decision inputs: probabilities, intervals, expected losses, and risk thresholds.

Model + data $\rightarrow$ posterior draws $\rightarrow$ better decisions.